

Aspect-Based Knowledge Graph Retrieval for Multi-Hop Question Answering

Tanuj Kanani
DA-IICT
Gandhinagar, India
202301474@daiict.ac.in

Chaitanya Vats
DA-IICT
Gandhinagar, India
202301419@daiict.ac.in

Abstract

Retrieval-Augmented Generation (RAG) pipelines typically represent each document chunk with a single dense embedding, which conflates multiple topics within a chunk and limits retrieval precision for multi-hop questions. We propose Aspect-KGI, a pipeline that replaces flat chunk embeddings with multiple *aspect centroids* derived by clustering the knowledge-graph triplets extracted from each chunk. At query time, the question is also decomposed into triplets, and each query triplet is matched against chunk aspects rather than a single vector. Combined with hybrid seed selection and multi-hop graph traversal, this approach improves answer quality on both HotpotQA and MuSiQue benchmarks. On 1000 questions per dataset, Aspect-KGI raises the average GPT-4o-mini judge score from 5.19 to 5.98 on HotpotQA and from 4.37 to 5.21 on MuSiQue compared to the baseline KGI system, with consistent gains across all five evaluation dimensions.

1 Introduction

Standard RAG pipelines work by embedding document chunks into a vector space and retrieving the top- k chunks closest to the query embedding. This single-vector representation works well for single-hop factual lookups, but it struggles with multi-hop questions where the answer depends on connecting facts from different parts of the corpus. A chunk that is semantically close to the query may still lack the specific relation needed to answer the question, while a distant chunk holding the missing piece of the reasoning chain is never retrieved.

Knowledge Graph (KG) based retrieval addresses this partially by extracting entity-relation-entity triplets from documents and building a graph that can be traversed to follow reasoning chains across chunks. However, the initial step of selecting which chunks to consider still relies on flat cosine similarity in most existing KG-RAG pipelines. If the right chunks are not selected in this first stage, the graph traversal has no way to recover the missing information.

We propose a simple but effective change to this first stage: instead of representing each chunk with a single embedding, we represent it with multiple *aspect centroids*. Each aspect is the centroid of a cluster of semantically related triplets within that chunk. At query time, the question is also decomposed into triplets, and each query triplet is matched against the chunk’s aspects individually. This gives the retriever a finer-grained signal, a chunk scores high only if it has aspects that specifically match the informational needs of the question, not just if it is broadly about the same topic.

2 Problem Definition

Given a corpus of text documents and a natural-language question that may require reasoning over two or more passages (multi-hop QA), the task is to:

- (1) Retrieve the most relevant document chunks from the corpus.
- (2) Traverse a knowledge graph built from the retrieved chunks to collect supporting facts.
- (3) Generate a concise, factual answer grounded in the collected facts.

We evaluate on two established multi-hop QA benchmarks, HotpotQA [1] and MuSiQue [2], each with 1000 question-answer pairs. We compare our aspect-based retrieval method (Aspect-KGI) against a baseline that uses plain cosine-similarity chunk retrieval (KGI), keeping all other components identical to isolate the effect of the retrieval change.

3 Method

Our pipeline has two phases: an offline indexing phase that processes documents once, and an online query phase that runs for each incoming question.

3.1 Offline Phase: Indexing

Step 1 – Chunking. Documents are split into chunks at heading boundaries (#, ##, ###). Adjacent small sections are merged greedily up to a maximum of 4096 characters. Any remaining oversized chunk is further split with 512-character overlap to avoid losing context at boundaries.

Step 2 – Triplet extraction. Each chunk is processed by Qwen-2.5 14B (running locally via Ollama at temperature 0) through a three-pass pipeline:

- (1) *Coreference resolution*: all pronouns and references are replaced with explicit entity names so that downstream triplets are self-contained.
- (2) *Atomic fact extraction*: the resolved text is broken into one discrete fact per line.
- (3) *Triplet generation*: each fact is converted into a Subject | Relation | Object triplet.

Step 3 – Aspect building. The triplet strings from each chunk are embedded using the all-MiniLM-L6-v2 sentence-transformer (with L2 normalisation). These embeddings are clustered using an adaptive DBSCAN procedure: cosine-distance epsilon is swept from 0.10 to 0.90 in steps of 0.05, and the sweep stops just before the cluster count drops by more than 40% in a single step (the “burst-drop” criterion). This avoids both over-fragmentation and over-merging. Each cluster’s mean embedding is L2-normalised to produce a unit-norm centroid. The set of centroids for a chunk constitutes its aspects.

Step 4 – Graph construction. All triplets across all chunks are inserted into a NetworkX directed multigraph. Both subject and object are added as nodes, with edges carrying the relation and the source chunk ID. This graph is shared across all queries.

3.2 Online Phase: Query Processing

Step 1 – Query decomposition. The incoming question goes through the same three-pass LLM pipeline (coreference → fact extraction → triplet generation) to produce a set of query triplets. If the LLM fails to produce parseable triplets, the raw query string is used as a fallback.

Step 2 – Aspect-based chunk retrieval. Each query triplet is embedded with the same sentence-transformer. For each chunk, the system computes the cosine similarity between every query triplet embedding and every aspect centroid of that chunk. A chunk’s score is the sum of the best-matching aspect similarities that exceed a threshold of 0.40. The top-50 chunks by this score are selected.

Step 3 – Hybrid seed selection. Candidate seed entities for graph traversal are drawn from two sources: entities mentioned in the query triplets (query-side signal) and entities that appear frequently across the top-50 retrieved chunks (corpus-side signal). Each candidate is scored with a weighted combination:

$$\text{score}(e) = \alpha \cdot \text{sim}(e, q) + \beta \cdot \text{freq}(e) + \gamma \cdot \text{bonus}(e) \quad (1)$$

where $\text{sim}(e, q)$ is the cosine similarity between the entity string embedding and the query embedding, $\text{freq}(e)$ is the entity’s normalised frequency across the top-50 chunks, and $\text{bonus}(e) = 1$ if the entity appears in the query triplets. The weights are $\alpha = 2.0$, $\beta = 1.5$, $\gamma = 3.0$. The top-10 entities by this score are selected as seeds.

Step 4 – Graph traversal. A 3-hop breadth-first search is run from each seed entity on the full graph (not restricted to the top-50 chunks). At each hop, all triplet texts along traversed edges are collected. This allows the system to discover facts from chunks that were never in the initial retrieval set, following reasoning chains across documents.

Step 5 – Triplet re-ranking. The collected triplets are deduplicated and then re-ranked in two stages: first by TF-IDF similarity to the query (keeping 2× the final budget as candidates), then by bi-encoder cosine similarity using the sentence-transformer (keeping the top 120). This mirrors a coarse-to-fine filtering approach.

Step 6 – Answer generation. The top-ranked triplets are formatted as context and passed to Qwen-2.5 14B with a prompt that instructs the model to answer strictly from the provided facts. If the facts are insufficient, the model is instructed to say so rather than hallucinate.

4 Baseline: KGI

The baseline system (KGI) uses the same graph construction, 3-hop BFS traversal, and LLM answer generation, but differs in the retrieval step. Instead of aspect-based scoring, KGI computes a single dense embedding per chunk (encoding the full chunk text) and retrieves the top- k chunks by cosine similarity to the query embedding. It does not decompose the query into triplets and does

Table 1: GPT-4o-mini judge scores (0–10) averaged over 1000 questions per benchmark. Δ shows the improvement of Aspect-KGI over KGI.

Metric	HotpotQA			MuSiQue		
	KGI	Aspect	Δ	KGI	Aspect	Δ
Completeness	5.07	5.94	+0.87	4.16	5.01	+0.85
Accuracy	5.35	6.15	+0.80	4.51	5.42	+0.91
Knowledgeability	5.23	6.00	+0.77	4.44	5.34	+0.90
Relevance	5.23	6.03	+0.80	4.40	5.18	+0.78
Coherence	5.08	5.79	+0.71	4.36	5.12	+0.76
Average	5.19	5.98	+0.79	4.37	5.21	+0.84

not cluster chunk triplets into aspects. Both vector search and TF-IDF search are used to find seed entities, which are then passed to the same multi-hop traversal and answer generation pipeline.

This setup ensures that the only variable being tested is the retrieval mechanism — all downstream components are identical.

5 Evaluation

5.1 Datasets

We evaluate on two multi-hop QA benchmarks:

- **HotpotQA** [1]: bridge-type questions requiring two Wikipedia paragraphs to answer. We use 1000 question-answer pairs.
- **MuSiQue** [2]: questions requiring 2–4 reasoning hops composed from single-hop sub-questions. We use 1000 question-answer pairs.

5.2 Evaluation Protocol

Each predicted answer is evaluated by GPT-4o-mini as an external judge across five rubric dimensions, each scored on a 0–10 scale:

- **Completeness:** whether the answer includes all important facts from the ground truth.
- **Accuracy:** whether the answer is factually correct.
- **Knowledgeability:** whether the answer shows domain knowledge consistent with the ground truth.
- **Relevance:** whether the answer stays on-topic.
- **Logical coherence:** whether the answer is clear and logically structured.

We also run a separate binary semantic judge using Qwen-2.5 14B with a fast-match fallback (exact normalised match, yes/no match, and containment check with negation detection) for correct/incorrect classification.

5.3 Models

- **LLM:** Qwen-2.5 14B via Ollama (temperature 0, 8192 context window) for triplet extraction, answer generation, and local binary judging.
- **Embeddings:** all-MiniLM-L6-v2 sentence-transformer for all triplet and query embeddings.
- **External Judge:** GPT-4o-mini for the five-rubric evaluation.

6 Results

Table 1 shows the GPT-4o-mini judge scores for both systems on both benchmarks.

6.1 Analysis

Aspect-KGI outperforms the baseline KGI on every metric across both benchmarks. On HotpotQA, the average score rises from 5.19 to 5.98 (+0.79), and on MuSiQue from 4.37 to 5.21 (+0.84). Several observations stand out:

Accuracy sees the largest gains. The accuracy improvement on HotpotQA is +0.80 and on MuSiQue is +0.91, the largest single-metric gain. This confirms that aspect-based retrieval is surfacing more factually relevant triplets for the answer generation step, directly reducing hallucination and incorrect facts.

MuSiQue benefits more. MuSiQue questions require 2–4 reasoning hops, making them inherently harder. The average improvement on MuSiQue (+0.84) is slightly larger than on HotpotQA (+0.79), suggesting that aspect-based retrieval is particularly helpful when the question has multiple distinct informational needs that must each be matched to different chunks.

Gains are consistent. The improvement is not concentrated on a single dimension. All five metrics improve on both benchmarks, indicating that the benefit extends from factual correctness to logical structure and completeness.

Coherence improves despite being downstream. Logical coherence is a property of the generated text, not directly of the retrieval. The fact that it improves (+0.71 on HotpotQA, +0.76 on MuSiQue) suggests that providing the LLM with better-selected, more relevant facts leads to more coherent reasoning in the generated answer.

7 Discussion

7.1 Why Aspects Help

A single embedding for a chunk is forced to represent all the information in that chunk as one point in vector space. If a chunk discusses both solar panel installation procedures and grid connection regulations, its embedding will be some average of these two topics. A query specifically about installation procedures may rank this chunk lower than a chunk that is entirely about procedures but lacks the specific fact needed.

By splitting the chunk’s triplets into aspect clusters, each centroid captures one coherent semantic theme. The query triplet about installation procedures can match the installation aspect with high similarity, regardless of what other topics the chunk contains. This is especially valuable for multi-hop questions, where different parts of the question may need to match different aspects of different chunks.

7.2 Adaptive DBSCAN

The adaptive epsilon sweep avoids the need to pre-specify the number of aspects per chunk. A topically diverse chunk naturally produces more aspects (the cluster count remains high as epsilon increases), while a focused chunk converges to fewer aspects (clusters merge quickly). The burst-drop stopping criterion (40% threshold) provides a principled way to detect when communities are being merged too aggressively, stopping the sweep at the last stable clustering.

7.3 Limitations

The three-pass LLM decomposition at query time adds latency compared to a simple embedding lookup. Each query requires three sequential LLM calls (coreference, fact extraction, triplet generation) before retrieval can begin. In the offline pipeline, triplet extraction for a large corpus can take several hours since every chunk must be processed through the same three passes.

The hybrid seed scoring formula uses fixed weights (α , β , γ) that were chosen based on initial experimentation. These weights were not systematically tuned, and their optimal values may differ across domains and datasets.

7.4 Future Work

Future work could explore distilling the three-pass query decomposition into a single smaller model trained to produce triplets directly, reducing online latency. Systematic hyperparameter tuning of the seed scoring weights and the DBSCAN parameters could further improve performance. Extending the evaluation to single-hop benchmarks would help quantify whether aspect-based retrieval also helps (or hurts) simpler questions where a flat embedding may be sufficient.

References

- [1] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [2] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the ACL*, 10:539–554, 2022.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in NeurIPS*, 2020.
- [4] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of KDD*, 1996.
- [5] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP*, 2019.